

Assignment 2: Classification Report

Chengze Liu 2316609

Task 1: K-NN classifier

B. In the report, describe the values of model parameters (such as metric, weights, etc.) that are set to be different from the default settings.

The parameters I set different from default in `sklearn.model_selection.StratifiedKFold()` are: (1) `shuffle = True`, this prevents unbalanced split in case the data is ordered; (2) `random_state = 1`.

The parameters I set different from default in `sklearn.neighbors.KNeighborsClassifier()` is: `metric = "euclidean"`.

D. Perform 5-fold cross-validation for each of the 4 neighbor sizes and compute accuracy, precision, recall, macro-F1 and micro-F1. These metrics are average of the 5-fold validation sets. In each fold, precision and recall are averaged for all classes. Generate a table, as given below, for the obtained results.

Table 1. K-NN Experiments

Neighbors	Normalization	Accuracy	Precision	Recall	Macro-F1	Micro-F1
2	No	0.7870	0.7395	0.7590	0.7470	0.7870
	Yes	0.9110	0.8668	0.8789	0.8714	0.9110
5	No	0.8042	0.7683	0.7344	0.7471	0.8042
	Yes	0.9280	0.9136	0.8824	0.8953	0.9280
10	No	0.8038	0.7730	0.7327	0.7471	0.8038
	Yes	0.9310	0.9168	0.8895	0.9010	0.9310
50	No	0.7448	0.7284	0.6490	0.6677	0.7448
	Yes	0.9122	0.9029	0.8525	0.8705	0.9122

E. Explain how the number of neighbors affects model performance in the context of overfitting/underfitting.

When the number of neighbors is low, for example, $k=2,3,4$, the model looks at very few neighbors to make a prediction. The model has low bias and high variance. The model is

able to fit any local pattern but, in the meantime, very sensitive to noise. A little variation in the testing set may yield completely different predictions. These are the characteristics of overfitting.

When the number of neighbors is high, for example, $k=30,40,50$, the model looks at a lot of neighbors to make a prediction. The model has high bias and low variance. When the sample size grows bigger in a majority rule, the model becomes less sensitive to noise and variations in the testing set. However, it also oversimplifies the model and cannot reveal local patterns. These are the characteristics of underfitting.

F. Describe how the normalization affects model performance and explain why normalization makes a difference.

All metrics had an increase after normalization regardless of the number of neighbors k . This is because KNN is a distance-based model and distance can be strongly influenced by some features with larger numerical scales. Whether two data points are neighbors may be largely determined by these features, and features with smaller scales and their underlying pattern may be missed. To address this, normalization puts all features under comparable scales. Features equally contribute to the distance and produce a more unbiased KNN model.

In addition, $k=50$ saw the largest improvement of 22% in accuracy, while $k=2$, $k=5$, $k=10$ see 15%. This strengthens the idea of distance and scale since features with larger scale are again magnified by the increased number of neighbors.

G. Explain the meaning of the difference in accuracy, precision and recall scores in relation to the task; only if there is a significant difference.

Let's first look at the difference between groups with different k .

Before normalization, $k=5$ and $k=10$ have the highest scores for accuracy, precision, and recall. $k=2$ has lower scores, and $k=50$ has the lowest. This is because $k=2$ overfits and $k=50$ underfits, as explained before.

After normalization, the scores for accuracy, precision, and recall are similar across all k . This impact of normalization is explained before.

Then let's look at the difference for a single k , before and after normalization.

There is a significant increase in accuracy after normalization for all k, meaning the overall proportion of correctly classified samples improved after normalization.

There is a significant increase in precision after normalization for all k, meaning that a prediction the model made would be more likely to be correct. In other words, fewer false positives.

There is a significant increase in recall after normalization for all k, meaning that the model is better at identifying all instances of a class. In other words, fewer false negatives.

Overall, it means that normalization significantly improves the performance of the KNN model.

Task 2: Decision Tree

B. In the report, describe the values of model parameters (such as gini/entropy criterion, min_samples_split, etc.) that are set to be different from the default settings.

The parameters I set different from default in `sklearn.model_selection.StratifiedKFold()` are: (1) `shuffle = True`, this prevents unbalanced split in case the data is ordered; (2) `random_state = 1`.

The parameters I set different from default in `sklearn.tree.DecisionTreeClassifier()` are: (1) `criterion = "entropy"`; (2) `min_samples_split = 4`, to reduce overfitting; (3) `random_state = 1`.

C. Perform 5-fold cross-validation for each of the 5 max depths and compute accuracy, precision, recall, macro-F1 and micro-F1. These metrics are average of the 5-fold validation sets. In each fold, precision and recall are averaged for all classes. Generate a table, as given below, for the obtained results.

Table 2. Decision Tree Experiments

Max Depths	Accuracy	Precision	Recall	Macro-F1	Micro-F1
3	0.8860	0.8320	0.8212	0.8244	0.8860
5	0.9066	0.8799	0.8482	0.8582	0.9066
10	0.9274	0.8994	0.8922	0.8955	0.9274
30	0.9202	0.8865	0.8862	0.8861	0.9202
50	0.9202	0.8865	0.8862	0.8861	0.9202

D. Explain how the tree depth affects model performance in the context of overfitting/underfitting.

When tree depth is low, for example, depth=3, the decision tree may be too simple and does not capture local patterns. The model is more tolerant to noise, showing low variance and high bias. This is underfitting.

When tree depth is high, for example, depth=50, the decision tree tries too hard to fit small local pattern in the training set. Consequently, any small variation in a testing set may lead to completely different predictions. This shows low bias and high variance and is the sign of overfitting.

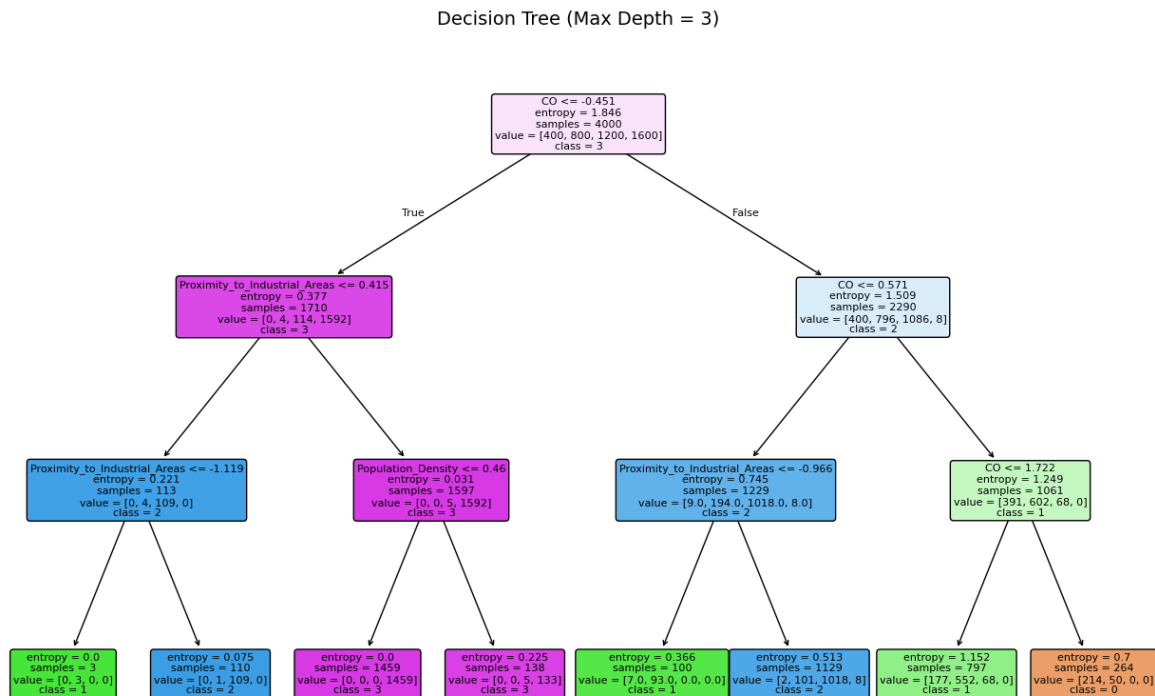
E. Explain the meaning of the difference in accuracy, precision and recall scores in relation to the task; only if there is a significant difference.

The decision tree has a boost in all metrics as its maximum depth increases and reaches the best performance at depth=10. At depth=30 and depth=50 the model sees a very slight drop in performance.

This is because at depth=3 and depth=5 the decision tree is underfitting. As the depth grows, the decision tree learns more complex feature relationships and captures more underlying patterns. At depth=10, the model reaches a balance between underfitting and overfitting and performs the best. Beyond this depth, the model starts to overfit. The complexity grows without improvement in any accuracy score. We see no difference in the metrics at depth=30 and depth=50.

F. Plot the Decision Tree with depth 3 and interpret the decision process according to the tree structure (e.g., consider gini/entropy gain, features, split, class of each node, paths from root to leaf nodes, etc.)

Figure 1: Decision Tree with depth=3



root:

- split condition: $CO \leq -0.451$
- entropy: 1.846
- majority is class 3 (good)
- interpretation: entropy is very high before classification. The first split is CO, meaning that CO has presumably the highest impact on air quality.

depth=1

left child:

- path from root: $CO \leq -0.451$
- split condition: $Proximity_to_Industrial_Areas \leq 0.415$
- entropy: 0.377
- majority is class 3 (good)
- interpretation: entropy is much lower than root. This node looks pure on class 3.

right child:

- path from root: $CO > 0.451$
- split condition: $CO \leq 0.571$

- entropy: 1.509
- majority is class 2 (moderate)
- interpretation: entropy is still very high after previous split (root). The next split keeps working on CO. This supports the speculation that CO has a big impact on air quality.

depth=2

left subtree 1:

- path from root: $CO \leq -0.451$, $Proximity_to_Industrial_Areas \leq 0.415$
- split condition: $Proximity_to_Industrial_Areas \leq -1.119$
- entropy = 0.221
- majority is class 2 (moderate)
- interpretation: entropy is lower than the parent's and majority changes from class 3 (parent) to class 2. Solid progress in classifying.

right subtree 1:

- path from root: $CO \leq -0.451$, $Proximity_to_Industrial_Areas > 0.415$
- split condition: $Population_Density \leq 0.46$
- entropy: 0.031
- majority is class 3 (good)
- interpretation: entropy is much, much lower than parent's and majority stay the same (class 3). The previous split did a very good job separating the classes.

left subtree 2

- path from root: $CO > 0.451$, $CO \leq 0.571$
- split condition: $Proximity_to_Industrial_Areas \leq -0.966$
- entropy: 0.745
- majority is class 2 (moderate)
- interpretation: the entropy is still high. Needs more depth and splits to classify.

right subtree 2

- path from root: $CO > 0.451$, $CO > 0.571$
- split condition: $CO \leq 1.722$
- entropy: 1.249
- majority is class 1 (poor)

- interpretation: the entropy is still very high. The next split is still on CO. It again highlights CO's importance.

depth=3

leaf 1:

- path from root: $CO \leq -0.451$, $Proximity_to_Industrial_Areas \leq 0.415$, $Proximity_to_Industrial_Areas \leq -1.119$
- entropy = 0
- predicts class 1 (poor)
- interpretation: with entropy of 0 and a sample size of 3, this leaf node seems to be fitting noise.

leaf 2:

- path from root: $CO \leq -0.451$, $Proximity_to_Industrial_Areas \leq 0.415$, $Proximity_to_Industrial_Areas > -1.119$
- entropy = 0.075
- predicts class 2 (moderate)
- interpretation: a very low entropy. This leaf node excels in classifying for class 2.

leaf 3:

- path from root: $CO \leq -0.451$, $Proximity_to_Industrial_Areas > 0.415$, $Population_Density \leq 0.46$
- entropy = 0
- predicts class 3 (good)
- interpretation: Both child nodes give the same prediction. This split was unnecessary.

leaf 4:

- path from root: $CO \leq -0.451$, $Proximity_to_Industrial_Areas > 0.415$, $Population_Density \leq 0.46$
- entropy = 0.225
- predicts class 3 (good)
- interpretation: Both child nodes give the same prediction. This split was unnecessary.

leaf 5:

- path from root: $CO > 0.451$, $CO \leq 0.571$, $Proximity_to_Industrial_Areas \leq -0.966$

- entropy = 0.366
- predicts class 1 (poor)
- interpretation: fairly low entropy but a sample size of 100 means there would be space for improvement had we increased depth

leaf 6:

- path from root: CO > 0.451, CO ≤ 0.571, Proximity_to_Industrial_Areas > -0.966
- entropy = 0.513
- predicts class 2 (moderate)
- interpretation: fairly high entropy. The classification can improve if we allow for more depth.

leaf 7:

- path from root: CO > 0.451, CO > 0.571, CO ≤ 1.722
- entropy = 1.152
- predicts class 1 (poor)
- interpretation: very high entropy: CO alone cannot resolve this classification. We need more depth and splits over other features to make better predictions.

leaf 8:

- path from root: CO > 0.451, CO > 0.571, CO > 1.722
- entropy = 0.7
- predicts class 0 (hazardous)
- interpretation: still very high entropy: CO alone cannot resolve this classification. We need more depth and splits over other features to make better predictions.

Task 3: SVM

B. In the report, describe the values of model parameters that are set to be different from the default settings.

The parameters I set different from default in `sklearn.model_selection.StratifiedKFold()` are: (1) `shuffle = True`, this prevents unbalanced split in case the data is ordered; (2) `random_state = 1`.

The parameters I set different from default in `sklearn.svm.SVC()` are: (1) `kernel = "linear"`, "poly", "rbf" or "sigmoid", as required; (2) `random_state = 1`.

C. Perform 5-fold cross-validation for each of the 4 kernel functions and compute accuracy, precision, recall, macro-F1 and micro-F1. These metrics are average of the 5-fold validation sets. In each fold, precision and recall are averaged for all classes. Generate a table, as given below, for the obtained results.

Table 3. SVM Experiments

Kernel Function	Accuracy	Precision	Recall	Macro-F1	Micro-F1
Linear	0.9414	0.9211	0.9099	0.9151	0.9414
Polynomial	0.9370	0.9220	0.9034	0.9114	0.9370
RBF	0.9456	0.9267	0.9160	0.9209	0.9456
Sigmoid	0.7564	0.6104	0.6041	0.6063	0.7564

D. Discuss the impact of different kernels on model performance.

Linear kernel performs very well. All metrics over 0.9. It is very likely air quality's classes are linearly separable with hyperplane decision boundaries.

Polynomial kernel performs slightly worse than linear kernel in terms of accuracy, recall, macrof1 and microf1. Polynomial kernel allows curved decision boundaries but also faces the risk of overfitting. Introducing higher dimensions into the dataset does not significantly improve the performance.

RBF kernel has the best performance, with all metrics topping the table. This imply that the data is mostly linearly separable but still has small non-linear patterns that RBF kernel can capture.

Sigmoid kernel performs significantly worse than the others. The reason may be that sigmoid kernel is not well-suited for multi-class classification.

E. Explain the meaning of the difference in accuracy, precision and recall scores in relation to the task; only if there is a significant difference.

Linear kernel and polynomial kernel have a relatively low recall, meaning that its performance on identifying the true classes out of the sample falls behind.

Sigmoid kernel has a significantly low precision and recall, meaning that the model often predicts class incorrectly and misses a lot of instances of each class.

Task 4: Bagging Classifier with Decision Tree

B. In the report, describe the values of model parameters that are set to be different from the default settings.

The parameters I set different from default in `sklearn.model_selection.StratifiedKFold()` are: (1) `shuffle = True`, this prevents unbalanced split in case the data is ordered; (2) `random_state = 1`.

The parameters I set different from default in `sklearn.tree.DecisionTreeClassifier()` are: (1) `criterion = "entropy"`; (2) `min_samples_split = 4`, to reduce overfitting; (3) `random_state = 1`.

The parameters I set different from default in `sklearn.ensemble.BaggingClassifier()` are: (1) `estimator = dt` (a `sklearn.tree.DecisionTreeClassifier` object), as required; (2) `n_estimators = 2, 5, 10, 100`, as required; (3) `random_state = 1`.

C. Perform 5-fold cross-validation for each of the 4 numbers of estimators and compute accuracy, precision, recall, macro-F1 and micro-F1. These metrics are average of the 5-fold validation sets. In each fold, precision and recall are averaged for all classes. Generate a table, as given below, for the obtained results.

Table 4. Bagging Classifier Experiments

Estimators	Accuracy	Precision	Recall	Macro-F1	Micro-F1
2	0.9142	0.8750	0.8945	0.8829	0.9142
5	0.9456	0.9243	0.9181	0.9210	0.9456
10	0.9476	0.9261	0.9207	0.9232	0.9476
100	0.9512	0.9318	0.9241	0.9277	0.9512

D. Discuss the impact of different numbers of estimators on model performance in the context of ensemble models.

As the number of estimators increases, the performance improves as all metrics see an increase. We see the greatest jump in scores from 2 estimators to 5 estimators and

improvement slows down from 5 estimators to 10 estimators and from 10 to 100. It is likely the borderline of diminishing returns is 5 estimators.

Task 5: Ensemble Models

C. In the report, describe the values of model parameters that are set to be different from the default settings.

The parameters I set different from default in `sklearn.model_selection.StratifiedKFold()` are: (1) `shuffle = True`, this prevents unbalanced split in case the data is ordered; (2) `random_state = 1`.

The parameters I set different from default in `sklearn.tree.DecisionTreeClassifier()` are: (1) `criterion = "entropy"`; (2) `min_samples_split = 4`, to reduce overfitting; (3) `random_state = 1`.

The parameters I set different from default in `sklearn.ensemble.BaggingClassifier()` are: (1) `estimator = dt` (a `sklearn.tree.DecisionTreeClassifier` object), as required; (2) `n_estimators = 100`, as required; (3) `random_state = 1`.

The parameter I set different from default in **`sklearn.ensemble.GradientBoostingClassifier()`** is: `random_state = 1`.

The parameter I set different from default in `lightgbm.LGBMClassifier` is: `random_state = 1`.

D. Perform 5-fold cross-validation for each of the two classifiers and compute accuracy, precision, recall, macro-F1 and micro-F1. These metrics are average of the 5-fold validation sets. In each fold, precision and recall are averaged for all classes. Generate a table, as given below, for the obtained results, and copy the results of Bagging Classifier with 100 estimators from Table 4.

Table 5. Boosting Classifier Experiments

Ensemble Model	Accuracy	Precision	Recall	Macro-F1	Micro-F1
Bagging	0.9512	0.9318	0.9241	0.9277	0.9512
Gradient Boosting	0.9558	0.9415	0.9316	0.9363	0.9558
LGBM	0.9554	0.9396	0.9302	0.9345	0.9554

E. Compare the three ensemble models and discuss which model is the best and the reason for its superiority.

All three ensemble models perform very well, with all metrics over 0.9. In terms of scores, the first is gradient boosting, second is lightgbm, and third is bagging, although the difference is very small.

Bagging with decision tree has slightly lower accuracy, precision, recall, macro-F1 and micro-F1 than the other two. It is probably because the decision trees to be averaged are independent to each other. Any bias in the decision tree cannot be corrected and the final ensemble model may have high bias but low variance.

On the other hand, boosting models such as gradient boosting and lightgbm sequentially build models that correct the errors in the prior models. The final ensemble model is low in bias, but the dependent nature of this sequence makes it sensitive to noise and high in variance.

In this case, it is likely that there is not much noise in the dataset so boosting models emerged superior in this bias-variance trade-off.

Task 6.

A. Interpret the tables you generated in Tasks 1D, 2C, 3C by comparing the performance of the Decision Tree, K-NN, SVM models. Which model performs better? Why do you think that is the case?

Best KNN model: $k=10$

Best decision tree: depth=10

Best SVM model: RBF kernel

Among these three models, SVM model with RBF kernel achieves the highest accuracy, precision, recall, Macro-F1 and Micro-F1. This is likely due to the model's capability to capture both linear and smaller non-linear patterns. The way SVM tries to maximize margin around the decision boundaries offers leeway for variation in test set. Therefore, the model has good balance in its bias and variance.

On the other hand, decision tree may fit noise and be more sensitive to test set variance. KNN is sensitive to the choice for k . $k=10$ might not be the most ideal choice. The result may get better if we test more values for k .

B. Interpret the tables you generated in Tasks 2C and 4C by comparing the performance of Decision Tree and Bagging Classifier with Decision Tree. Which model performs better? Why do you think that is the case?

Best decision tree model: depth=10

best bagging with decision tree model: estimators=100

Bagging with decision tree has significantly better performance than a single decision tree, with all metrics about 0.03 higher.

A deep decision tree tends to fit noise and have high variance. However, bagging with decision trees averages the result of 100 decision trees and is likely to smooth out the variation. Therefore, bagging with decision trees has better performance than a single decision tree.